SOLUCIÓN PROPUESTA

Patrones de Diseño Creacionales

Sistema de Alquiler de Vehículos Eléctricos — AutoCar

INGENIERIA DE SOFWARE II

Andrés Camilo Valarezo Payan

Curso: Patrones de Diseño Orientado a Objetos
Taller Calificable — Grupos | 2026

1. ¿Cuál era el problema?

El taller planteó el diseño del sistema de AutoCar, una plataforma de alquiler de vehículos eléctricos e híbridos. El equipo de desarrollo identificó dos problemas concretos que debían resolverse aplicando patrones de diseño creacionales:

- **Problema 1 — Inventario de vehículos:** Gestionar una flota formada por Autos, Vans y Camiones almacenada en un array de objetos, con operaciones de agregar, buscar por placa y ordenar por autonomía.
- **Problema 2 — Contratos de alquiler:** Construir contratos de forma ordenada, con campos obligatorios (cliente, vehículo, plan) y accesorios opcionales (GPS, seguro, cargador portátil). Además, aplicar automáticamente un descuento del 15% cuando el contrato supere los 30 días de duración.

La pregunta central era: ¿qué patrones de diseño creacionales resuelven mejor cada uno de estos dos problemas sin generar código repetitivo ni difícil de mantener?

2. La Decisión: Dos Patrones, Dos Problemas

Se eligieron dos patrones creacionales complementarios, uno para cada escenario:

Escenario	Patrón Elegido	¿Por qué?
Escenario 1 — Inventario	Factory Method	Centraliza la creación de vehículos. El sistema no necesita saber qué clase concreta instanciar.
Escenario 2 — Contratos	Builder	Construye el contrato paso a paso con validaciones en el medio (el descuento del 15%).

Estos dos patrones no compiten entre sí: se complementan. Factory Method resuelve cómo se crean los vehículos; Builder resuelve cómo se arman los contratos.

3. Escenario 1 — Factory Method para el Inventario

3.1 ¿Qué problema resuelve?

Sin el patrón, el código del main tendría que conocer y usar directamente cada clase concreta de vehículo:

```
// Sin Factory Method: el main debe saber qué clase concreta usar
Vehiculo v1 = new Auto("ABC-001", 350);
Vehiculo v2 = new Van("XYZ-002", 200);
Vehiculo v3 = new Camion("LMN-003", 280);
```

Esto parece simple, pero tiene un problema real: si en el futuro se agrega un nuevo tipo de vehículo (por ejemplo, Moto o Bicicleta eléctrica), hay que buscar todos los lugares del código donde se crean vehículos y modificarlos uno por uno. Eso viola el principio Abierto/Cerrado de OOP.

Con Factory Method, toda esa lógica queda centralizada en una sola clase: VehiculoFactory. El resto del sistema simplemente solicita un vehículo por tipo:

```
// Con Factory Method: el main no sabe ni le importa qué clase se instancia
Vehiculo v1 = VehiculoFactory.crear("auto",   "ABC-001", 350);
Vehiculo v2 = VehiculoFactory.crear("van",    "XYZ-002", 200);
Vehiculo v3 = VehiculoFactory.crear("camion", "LMN-003", 280);
```

3.2 ¿Por qué NO Prototype?

> **Alternativa descartada:** El patrón **Prototype** clona objetos existentes en lugar de crearlos desde cero. Sería útil si hubiera plantillas reutilizables, como un 'auto urbano estándar' del que se hacen copias cambiando solo la placa. En AutoCar, cada vehículo nace desde cero con placa y autonomía únicas. No hay estado base compartido que justifique la clonación, por lo que Prototype no aporta ninguna ventaja aquí.

3.3 Estructura de clases — Analogía con el código del profesor

La jerarquía de clases sigue exactamente el mismo esquema del código base del profesor:

Código del Profesor	Código AutoCar	Rol en el Patrón
Converter_ (abstract)	Vehiculo (abstract)	Clase base — define la estructura común

Código del Profesor	Código AutoCar	Rol en el Patrón
ASCIIConverter_	Auto	Subtipo concreto 1 — auto eléctrico
PDFConverter_	Van	Subtipo concreto 2 — van híbrida
PostScriptConverter_	Camion	Subtipo concreto 3 — camión ligero
(no existía)	VehiculoFactory	Fábrica central — crea los vehículos

Vehiculo es una clase abstracta con dos atributos protegidos (placa y autonomia) y un método abstracto mostrarInfo(). Cada subclase implementa ese método a su manera. Gracias al polimorfismo, el inventario puede llamar vehiculo.mostrarInfo() sin importar si es un Auto, una Van o un Camion.

3.4 El Inventario con array de objetos

> **Requisito del PDF:** El taller exige explícitamente usar un array de objetos (Vehiculo[]), no una lista dinámica como ArrayList. La clase Inventario cumple este requisito con capacidad fija definida al crear el objeto.

Se implementaron cuatro operaciones sobre el array:

- **agregar(Vehiculo v):** inserta el vehículo en la siguiente posición disponible del array. Si el array está lleno, muestra una advertencia.
- **buscarPorPlaca(String placa):** recorre el array linealmente comparando cada placa con equalsIgnoreCase(). Retorna el objeto si lo encuentra, o null si no existe.
- **ordenarPorAutonomia():** implementa el algoritmo de burbuja directamente sobre el array de objetos. Intercambia referencias Vehiculo[] cuando el elemento actual supera en autonomía al siguiente. El array queda ordenado de menor a mayor.
- **mostrarTodos():** itera el array y llama vehiculos[i].mostrarInfo() polimórficamente. Java ejecuta automáticamente la versión correcta según el tipo real del objeto, sin ningún if ni switch.

```
// Ordenamiento burbuja sobre el array de objetos
public void ordenarPorAutonomia() {
    for (int i = 0; i < contador - 1; i++) {
        for (int j = 0; j < contador - 1 - i; j++) {
            if (vehiculos[j].getAutonomia() > vehiculos[j+1].getAutonomia()) {
                Vehiculo temp  = vehiculos[j];       // intercambiar
                vehiculos[j]   = vehiculos[j+1];     // referencias
                vehiculos[j+1] = temp;               // en el array
```

```
                    }
                }
            }
        }
```

4. Escenario 2 — Builder para los Contratos

4.1 ¿Qué problema resuelve?

Un contrato de alquiler tiene campos que dependen unos de otros y una regla de negocio que debe validarse en el momento correcto: si la duración supera los 30 días, se aplica un descuento del 15%. No es suficiente un constructor con todos los parámetros juntos, porque la validación debe ocurrir durante la construcción, no después.

El patrón Builder divide la construcción en cuatro pasos que el Director ejecuta siempre en el mismo orden obligatorio:

1. **buildCliente():** asigna el nombre del cliente o empresa al contrato.
2. **buildVehiculo():** asigna el vehículo que se va a alquilar.
3. **buildPlan():** define el tipo de plan y la duración. Aquí se aplica el descuento del 15% si la duración supera los 30 días.
4. **buildAccesorios():** agrega los accesorios opcionales (GPS, seguro, cargador portátil) según el plan.

> **Clave del patrón:** El paso 3 (buildPlan) es donde ocurre la validación del descuento, durante la construcción, no después. Esto garantiza que ningún contrato pueda existir con duración mayor a 30 días sin su descuento correspondiente. Esa lógica en el medio del proceso de construcción es exactamente el escenario para el que fue diseñado el patrón Builder.

4.2 ¿Por qué NO Abstract Factory?

> **Alternativa descartada:** El patrón **Abstract Factory** garantiza coherencia entre familias de objetos: por ejemplo, que un cliente Empresarial nunca reciba accesorios de un cliente Particular. Sería útil si el problema fuera evitar combinaciones inválidas entre planes y accesorios. Pero el problema real aquí es la construcción secuencial con validaciones en el medio. Abstract Factory crea familias de objetos completos de una vez y no tiene un mecanismo para inyectar lógica entre pasos de construcción. Por eso Builder es la elección correcta.

4.3 Los tres builders concretos

Se implementaron tres builders concretos, uno por cada tipo de plan, exactamente igual a como el profesor tenía tres Converters distintos (ASCII, PDF, PostScript):

Builder Concreto	Duración	Descuento	Accesorios incluidos
ContratoBasicoBuilder	7 días	0%	Solo GPS
ContratoEmpresarialBuilder	45 días	15%	GPS + Seguro + Cargador
ContratoVipBuilder	90 días	15%	GPS + Seguro + Cargador

Los planes Empresarial y VIP superan los 30 días, por lo que el descuento se aplica automáticamente dentro del paso buildPlan(). El Plan Básico tiene 7 días, no supera el umbral, y no recibe descuento.

```
// Dentro de ContratoEmpresarialBuilder.buildPlan():
@Override
public void buildPlan() {
    contrato.setPlan("Plan Empresarial");
    contrato.setDuracionDias(45);
    // REGLA DEL PDF: duracion > 30 dias -> descuento del 15%
    if (contrato.getDuracionDias() > 30) {
        contrato.setDescuento(0.15);
    } else {
        contrato.setDescuento(0);
    }
}
```

4.4 El Director — equivalente al Lector_ del profesor

El Director es el equivalente exacto del Lector_ en el código del profesor. Recibe cualquier ContratoBuilder sin importar si es básico, empresarial o VIP, y llama los cuatro pasos en el orden correcto. No sabe qué tipo concreto de builder recibe; solo orquesta los pasos:

Código del Profesor (Lector_)	Código AutoCar (Director)
Lector_(String tipo, Converter_ c)	Director()
parseInput() llama make___()	construir(builder) llama build___()
Recibe ASCIIConverter_ o PDFConverter_	Recibe Basico, Empresarial o Vip builder
No sabe qué Converter_ usa internamente	No sabe qué builder concreto recibe

```
public class Director {
    public Contrato construir(ContratoBuilder builder) {
        builder.crearContrato();      // inicializar el objeto vacio
        builder.buildCliente();       // paso 1
        builder.buildVehiculo();      // paso 2
        builder.buildPlan();          // paso 3 — valida descuento aqui
        builder.buildAccesorios();    // paso 4
        return builder.getContrato(); // retornar el contrato terminado
    }
```

```
}
```

4.5 La clase Contrato — el producto final

La clase **Contrato** es el objeto que el Builder construye. Todos sus campos son **private** con setters, lo que garantiza encapsulamiento OOP correcto. Los builders los modifican a través de los setters, nunca con acceso directo. El método mostrar() imprime toda la información del contrato, incluyendo el descuento cuando aplica.

Campo (private)	Descripción
String cliente	Nombre del cliente o empresa
Vehiculo vehiculo	Referencia al vehículo asignado
String plan	Nombre del plan (Básico, Empresarial, VIP)
int duracionDias	Duración del contrato en días
boolean gps	Accesorio opcional: GPS
boolean seguro	Accesorio opcional: seguro vehicular
boolean cargadorPortatil	Accesorio opcional: cargador portátil
double descuento	Porcentaje de descuento (0.15 = 15%)

5. ¿Qué imprime el programa al ejecutarse?

Al correr java EjecutarAutoCar en la terminal, la salida se divide en dos bloques correspondientes a los dos escenarios:

5.1 Escenario 1 — Inventario inicial y operaciones

```
>>> ESCENARIO 1: Inventario de Vehiculos (Factory Method)

=== Inventario AutoCar (5 vehiculos) ===
Auto electrico  | Placa: ABC-001 | Autonomia: 350 km
Van hibrida     | Placa: XYZ-002 | Autonomia: 200 km
Camion ligero   | Placa: LMN-003 | Autonomia: 280 km
Auto electrico  | Placa: QRS-004 | Autonomia: 420 km
Van hibrida     | Placa: TUV-005 | Autonomia: 180 km
==================================================

>> Buscar placa LMN-003:
Camion ligero   | Placa: LMN-003 | Autonomia: 280 km

>> Ordenados por autonomia (menor a mayor):
=== Inventario AutoCar (5 vehiculos) ===
Van hibrida     | Placa: TUV-005 | Autonomia: 180 km
Van hibrida     | Placa: XYZ-002 | Autonomia: 200 km
Camion ligero   | Placa: LMN-003 | Autonomia: 280 km
Auto electrico  | Placa: ABC-001 | Autonomia: 350 km
Auto electrico  | Placa: QRS-004 | Autonomia: 420 km
==================================================
```

El inventario se muestra primero en el orden en que se agregaron los vehículos. Luego se busca la placa LMN-003 exitosamente. Finalmente, el algoritmo burbuja reordena el array de menor a mayor autonomía: de 180 km (TUV-005) a 420 km (QRS-004).

5.2 Escenario 2 — Los tres contratos de alquiler

```
>>> ESCENARIO 2: Contratos de Alquiler (Builder)

------ Contrato AutoCar ------
Cliente   : Juan Perez
Plan      : Plan Basico
Duracion  : 7 dias
Accesorios: [GPS]
Vehiculo  : Auto electrico  | Placa: ABC-001 | Autonomia: 350 km
------------------------------

------ Contrato AutoCar ------
Cliente   : Empresa TechCar S.A.
```

```
Plan      : Plan Empresarial
Duracion  : 45 dias
Accesorios: [GPS] [Seguro] [Cargador]
Descuento : 15%  (duracion > 30 dias)
Vehiculo  : Camion ligero   | Placa: LMN-003 | Autonomia: 280 km
------------------------------

------ Contrato AutoCar ------
Cliente   : Maria Lopez VIP
Plan      : Plan VIP
Duracion  : 90 dias
Accesorios: [GPS] [Seguro] [Cargador]
Descuento : 15%  (duracion > 30 dias)
Vehiculo  : Auto electrico  | Placa: QRS-004 | Autonomia: 420 km
------------------------------
```

- **Plan Básico (7 días):** sin descuento, solo GPS. La duración no supera el umbral de 30 días.
- **Plan Empresarial (45 días):** descuento del 15% aplicado automáticamente. Todos los accesorios incluidos.
- **Plan VIP (90 días):** descuento del 15% aplicado automáticamente. Todos los accesorios premium incluidos.

6. Conclusión

La solución propuesta aplica dos patrones de diseño creacionales que se complementan perfectamente para resolver los dos escenarios del taller:

- **Factory Method** resuelve el problema de crear vehículos de forma centralizada y flexible. El sistema no necesita saber qué clase concreta instanciar; la fábrica lo decide internamente. Agregar un nuevo tipo de vehículo solo requiere crear una nueva clase y añadir una línea en la fábrica, sin tocar ningún otro archivo.

- **Builder** resuelve el problema de construir contratos de forma ordenada con lógica de negocio entre pasos. El Director garantiza que los contratos siempre se arman en el mismo orden, y cada builder concreto puede tener su propia lógica interna para el descuento y los accesorios. Agregar un nuevo tipo de contrato solo requiere un nuevo builder, sin modificar el Director ni la clase Contrato.

> **Lección principal:** Los patrones de diseño no son plantillas que se aplican mecánicamente. Se eligen según el problema específico: Factory Method cuando la creación varía pero el uso es uniforme; Builder cuando la construcción es secuencial con validaciones entre pasos. El código del profesor (Converter_ / Lector_) y el código AutoCar (Vehiculo / Director) son el mismo patrón aplicado a dos contextos distintos.